

BUILDER JOURNAL

AI Workspace Platform

1. Platform Design Decisions

The AI Workspace Platform was designed as a modular application with a clear end-to-end flow: registration and login → workspace → assistant → chat → persistence, with additional modules for Knowledge Base/RAG, Long-Term Memory, Prompt Library, AI Skills, Dashboard, and advanced features. A key decision was to prove one complete vertical slice before expanding to the remaining modules, so the core architecture could be verified early.

For AI provider selection, a single `get_provider(model)` chokepoint was introduced. Chat, skills, and document Q&A use this provider-resolution path. Model prefixes determine the provider, configured API keys enable real providers, and an unconfigured setup falls back to Mock. HashingVectorizer was used for embeddings because it is deterministic, offline, and has no paid API cost.

The frontend uses shared theme and component files to avoid duplicated styling. Documents were consolidated into Knowledge Base and Analytics into Dashboard rather than creating duplicate pages.

2. Biggest Architectural Challenge

The biggest challenge was making LLM provider selection reliable. Initially, the chat service directly instantiated `MockLLMProvider`, so the configured Gemini model and API key were ignored. Tracing the path from `.env` → Settings → Assistant Configuration → LLM Provider exposed the problem. Centralizing provider selection fixed it and was verified with and without an API key.

A second challenge appeared during Windows testing. A rate-limiting decorator combined with postponed annotations caused `FastAPI/Pydantic` to fail resolving `UserRegister` and the backend crashed. Removing postponed annotations from `auth.py` fixed the issue. This showed that passing tests in one environment does not guarantee identical behavior in another.

3. Memory Implementation

Long-Term Memory initially had a silent gap: memory items were stored in the database but were not passed to the LLM during chat. This was fixed by injecting pinned memory, followed by recent memory items, into the system prompt in `chat_service.py`. The behavior was verified with an automated test that inspected the system prompt sent to the provider. Memory supports adding, editing, pinning, unpinning, searching, deleting, and recalling information through chat.

4. Database Design

The database stores persistent state including users, workspaces, assistants, conversations, messages, documents, memory items, prompt templates, and workspace collaboration data. Workspace sharing required a `WorkspaceMember` table and share-token based access.

The design intentionally avoids unnecessary duplication. Bookmarks and pinned messages use the same `Message.pinned` mechanism, while Favorites specifically maps to Prompt Template favoriting. This keeps the schema simpler while satisfying the required functionality.

5. Lessons Learned

The project reinforced the importance of tracing real execution paths instead of assuming configuration is being used. It also showed why end-to-end testing should happen early and why manual testing can reveal issues automated tests miss. Streamlit's rerun model was especially important: the Use in Chat

bug occurred because session state was consumed too early, causing the send action to disappear. The fix retained the prefilled prompt until successful sending.

Another important lesson was environment-specific testing. The Windows Pydantic/slowapi issue did not reproduce in the Linux development environment, so testing in the actual user environment provided valuable feedback.

6. What I Would Improve

For a production version, I would replace the offline hashing-based embeddings with a stronger semantic embedding model or service, add more advanced workspace permission levels, and implement a persistent per-user theme preference instead of URL-based theme persistence. I would also execute the prepared deployment configuration and validate the application on the target hosting environment. Additional security, load, and cross-environment testing would strengthen production readiness.

7. Preparation for Week 6

For Week 6, the focus should be final verification and delivery: complete and document automated tests and evaluation scenarios, perform the required security and performance reviews, finalize screenshots and documentation, and complete deployment when hosting credentials are available. The functional audit has already identified and fixed important issues including Memory Recall, Workspace Management gaps, missing AI Skills, Use in Chat behavior, and Dark Mode persistence. The remaining work is focused on evidence, evaluation, deployment, and final polish.